

Reviewer Pack — Minimal Rank of Group Cocommutative Algebras Bounds BP Read-...

Entry bfceea7bd3d4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 10:29:44 UTC

Minimal Rank of Group Cocommutative Algebras Bounds BP Read-Twice Size

Entry ID: bfceea7bd3d4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 10:29:44 UTC

1. Conjecture

Field A (mathematical branch): Group Cocommutative Algebra **Field B** (complexity object): Complexity Theory: Branching Program Read-Twice Size

Statement:

[‘For every read-twice branching program P , the minimal rank of its group cocommutative algebra is $O(\log \text{size}(P))$.’, ‘Furthermore, for any inner product mod 2 (IP_2) trivial BP Q , the minimal rank of the group cocommutative algebra associated with Q is at least $\Omega(n)$.’]

Rationale (proposer’s reasoning):

[‘Group cocommutative algebras provide a natural setting to study non-commutative structures, which may reveal new insights into the complexity of branching programs.’, ‘The minimal rank as an invariant is computable for group algebras and can be linked to the size of the read-twice BP, potentially providing a new tool in understanding the complexity class NEXP.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 983a2cad45bab7a1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the group cocommutative algebra’s minimal rank, if the mean of the $\log(\text{size}(P))$ values is within ± 3 of the expected $O(\log \text{size}(P))$, and for inner product mod 2 trivial BP Q , the minimal rank is $\geq \Omega(n)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (1): - IP_2 trivial BP AND group cocommutative algebra minimal rank

Top relevant hits considered: - [<http://arxiv.org/abs/2212.07668v3>] BPS Lie algebras for totally negative 2-Calabi-Yau categories and nonabelian Hodge theory for stacks - [<http://arxiv.org/abs/1404.6053v1>] A ladder of topologically non-trivial non-BPS states - [<http://arxiv.org/abs/1112.0030v2>] Homological algebra of knots and BPS states

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        # Generate a read-twice branching program of size n
        bp = []
        for _ in range(n):
            node = {'inputs': [random.randint(0, 1)], 'outputs': []}
            for _ in range(random.randint(2, 4)):
                child_node = {'inputs': [random.randint(0, 1)], 'outputs': []}
                node['outputs'].append(child_node)
                child_node['parent'] = node
            bp.append(node)
        return bp

    def compute_group_cocommutative_algebra(bp):
        # Compute the group cocommutative algebra for a given BP
        # This is a placeholder function; actual implementation depends on the conjecture
        return 1.0 # Placeholder value

    def min_rank(algebra):
        # Calculate the minimal rank of the algebra
        # This is a placeholder function; actual implementation depends on the conjecture
        return len(algebra)

    n = random.randint(5, 40)
    bp = generate_read_twice_bp(n)
    algebra = compute_group_cocommutative_algebra(bp)
```

```

rank = min_rank(algebra)

log_size = math.log2(n) if n > 0 else float('inf')

return {
    "metric_name": "log_size_vs_min_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no_trials_run")
        exit(0)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08329b76.py", line 63, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08329b76.py", line 46, in run_trial
    rank = min_rank(algebra)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08329b76.py", line 41, in min_rank
    return len(algebra)
           ~~~~~^~~~~~
TypeError: object of type 'float' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16887
2	propose	ollama_remote	glm4:latest	0	0	11438
3	propose	ollama_remote	glm4:latest	0	0	9501
4	preregistration	ollama_remote	glm4:latest	0	0	5901
5	novelty	ollama_remote	glm4:latest	0	0	4729
6	novelty	ollama_remote	glm4:latest	0	0	5767
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14465
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8154
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7867
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7666
11	judge	ollama_remote	glm4:latest	0	0	13642

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106015 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/bfcee7bd3d4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bfcee7bd3d4.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bfcee7bd3d4.tar.gz (if generated)